Boosting

Albert Y. Kim

Recall

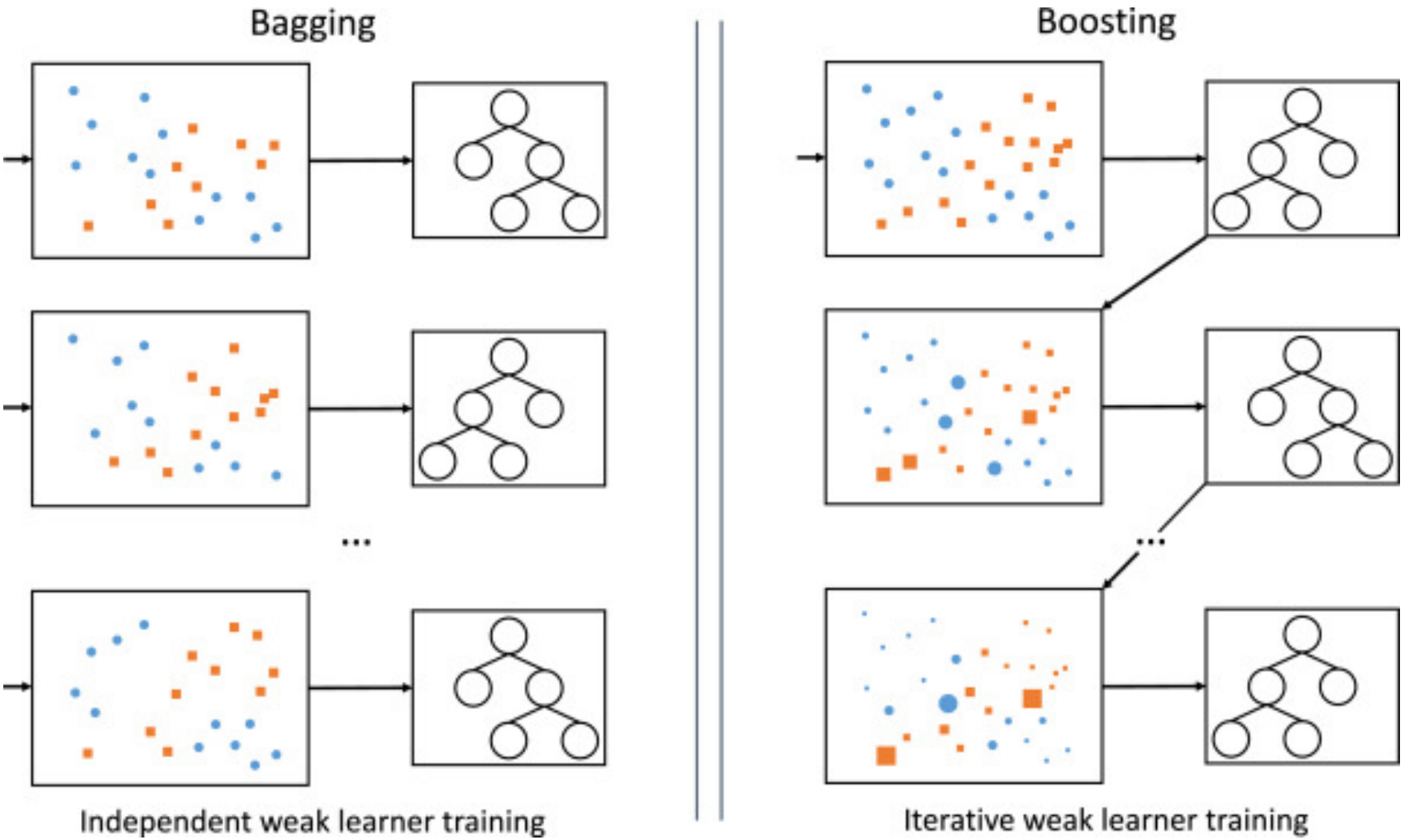
- Bagging (bootstrap aggregation) builds an ensemble model by aggregating individual base models. Each base model is trained on slightly different (bootstrap resampled with replacement) data
- Random Forests is a specific kind of bagging method where
  - The base models are trees
  - There is also a random sampling of a subset of the p features at each split in the tree

Today

- Random forests are an *aggregation* approach: base models in the ensemble are *trained separately* and the predictions are aggregated
- Today we will discuss *boosting* approaches: base models in the ensemble are *trained sequentially*
- Boosting was originally developed as a classifier aimed at combining many "weak" classifiers into a more powerful "committee" by Freund and Schapire (1997) as "Adaptive Boosting" or "AdaBoost.M1"

Bagging vs. Boosting

This image contrasts bagging with the *iterative and sequential* nature of boosting



AdaBoost

One such algorithm is AdaBoost. Suppose y is a binary encoded by $\{-1, 1\}$, and define the error rate as:

$$\text{error} = \frac{1}{n} \sum_{i=1}^{n} I(y_i \neq G(\mathbf{x}_i))$$

Here, $G(\mathbf{x}_i)$ represents the predicted class for an observation with predictors $\mathbf{x}_i$

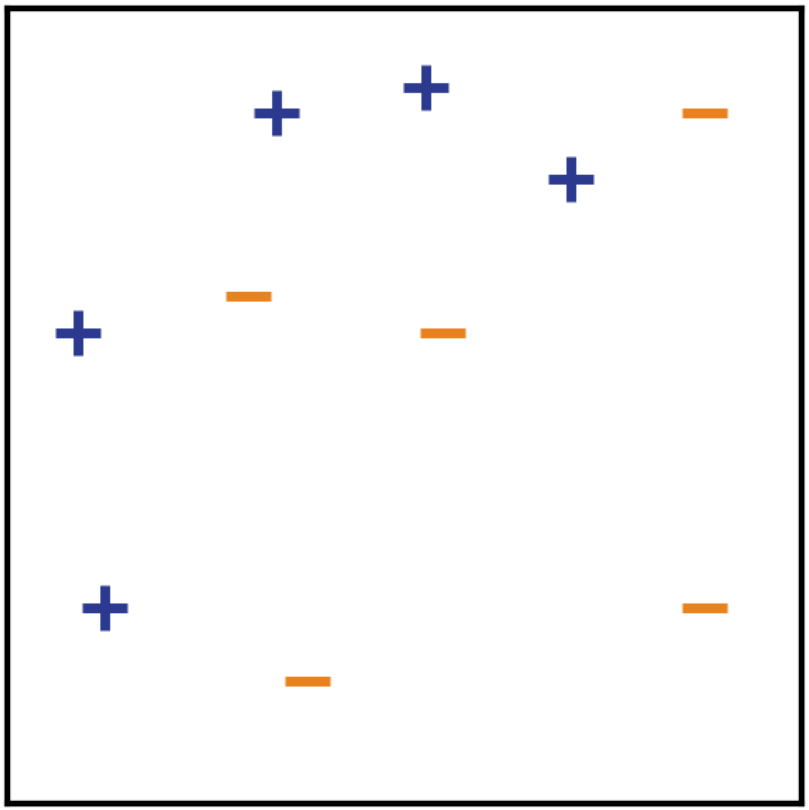
- Notice that if y_i does not match the predicted class the summation increases by 1
- Thus error $= 1 -$ **classification accuracy**

AdaBoost: Intution build-up

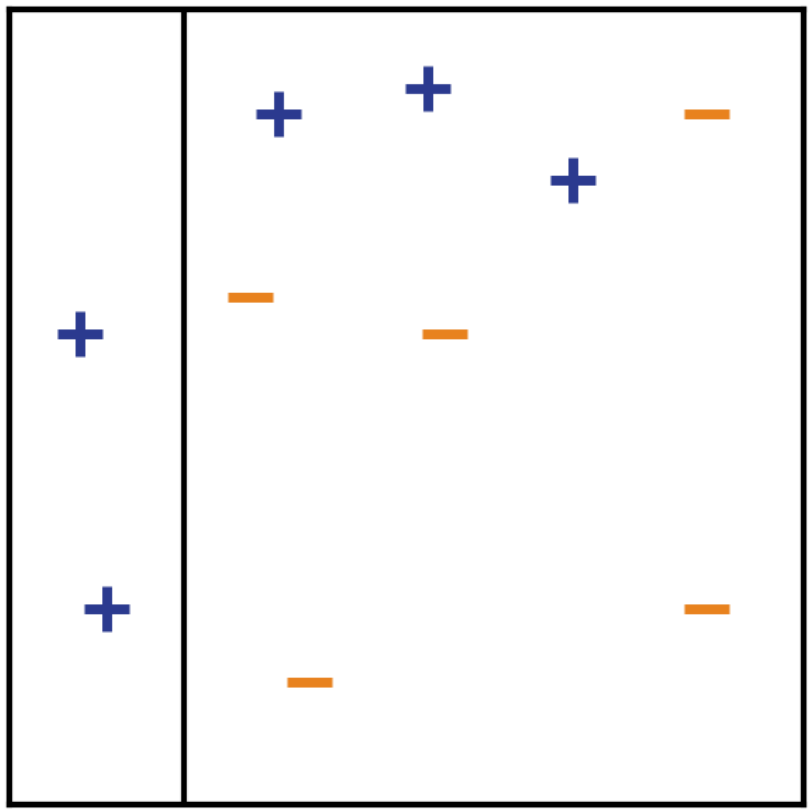
Let's build up the intuition behind AdaBoost with some help from:

- Statistics for Machine Learning by Pratap Dangeti
- Claude Sonnet 5.5, Low effort

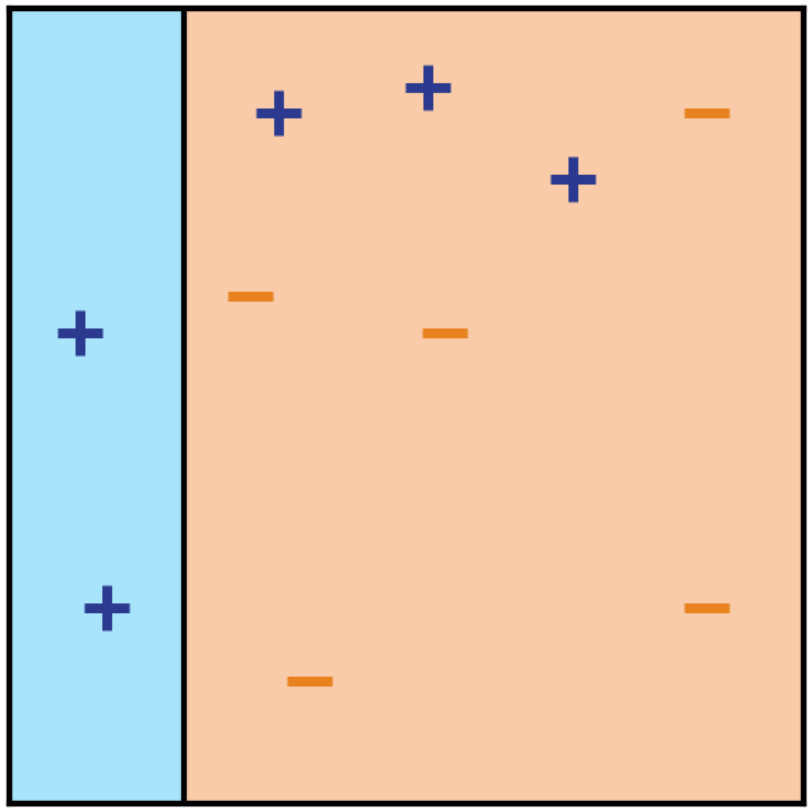
AdaBoost: Image 0



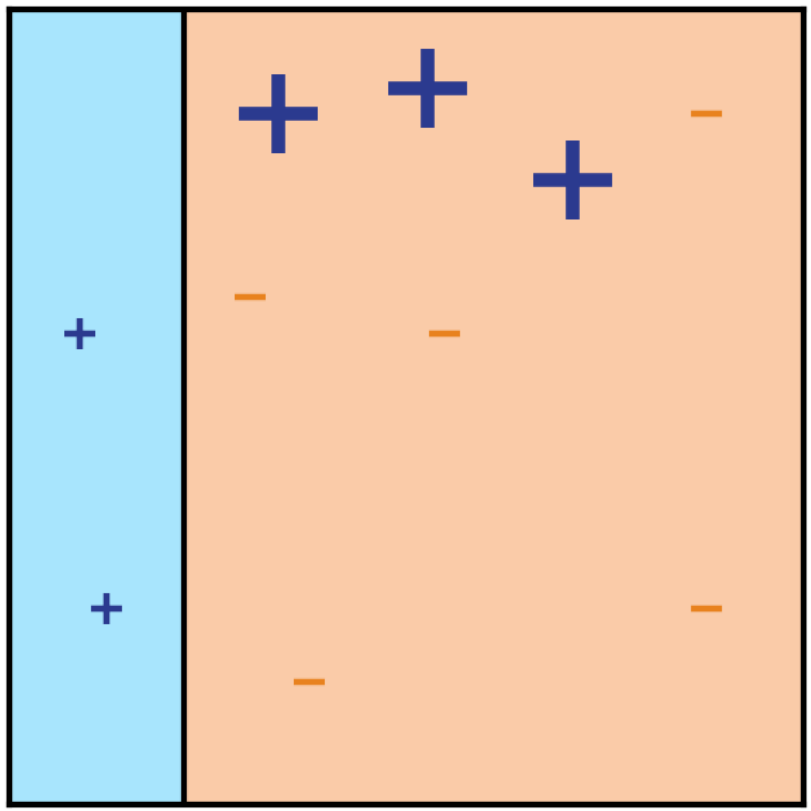
AdaBoost: Image 1



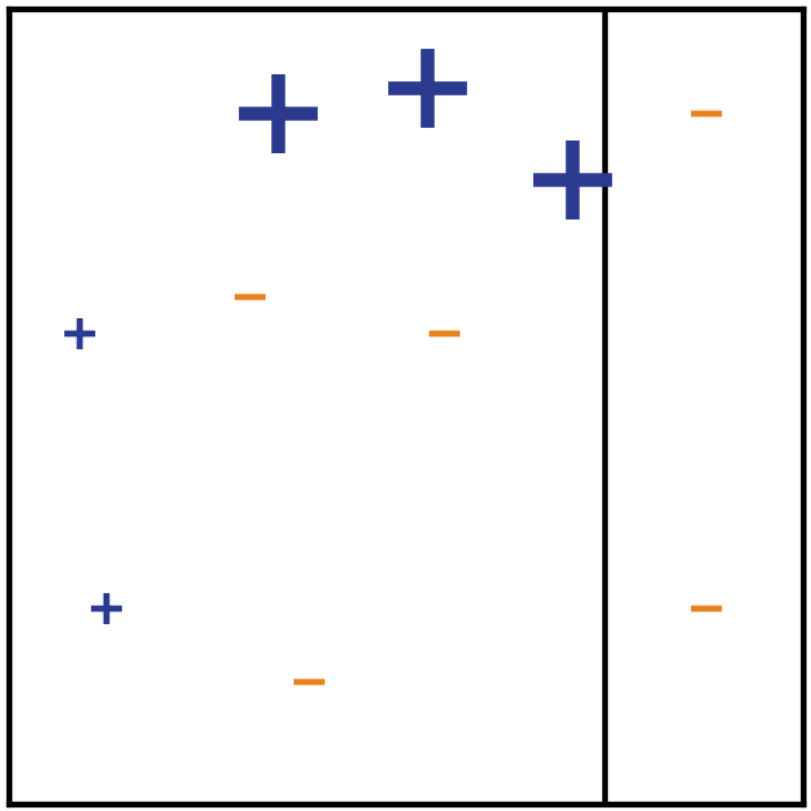
AdaBoost: Image 2



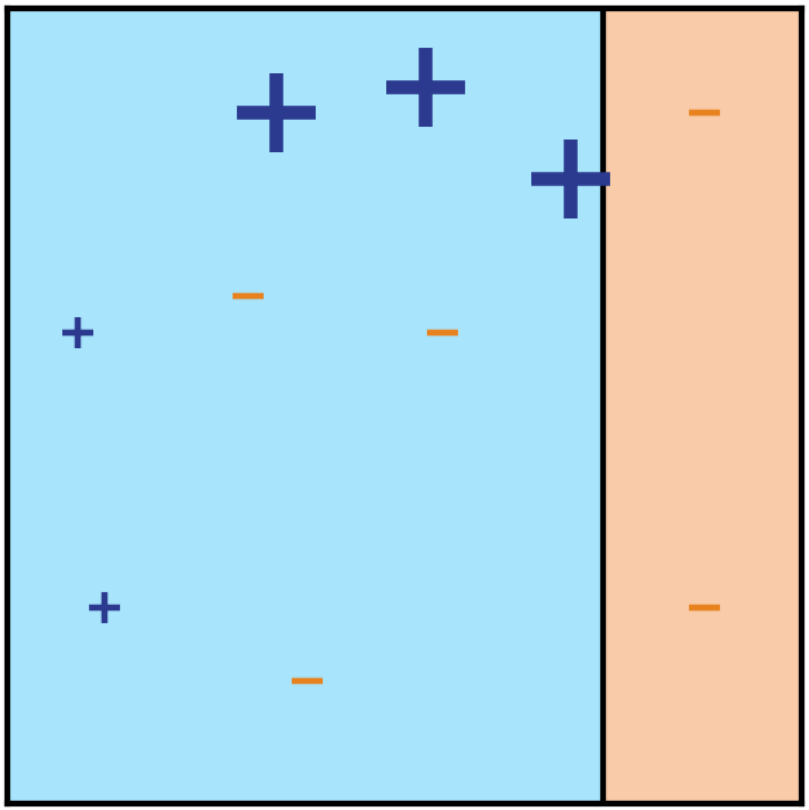
AdaBoost: Image 3



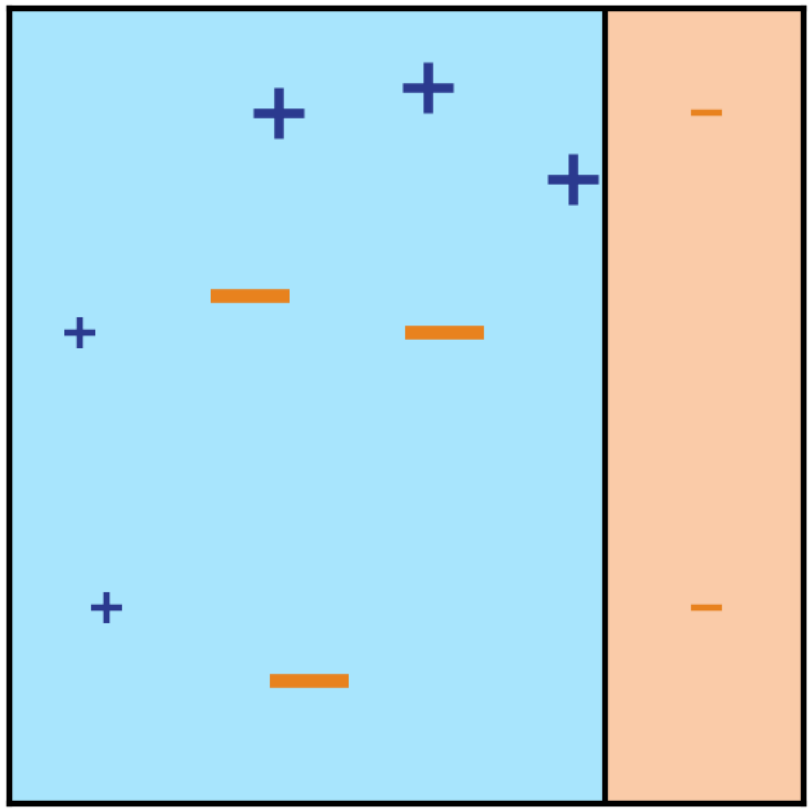
AdaBoost: Image 4



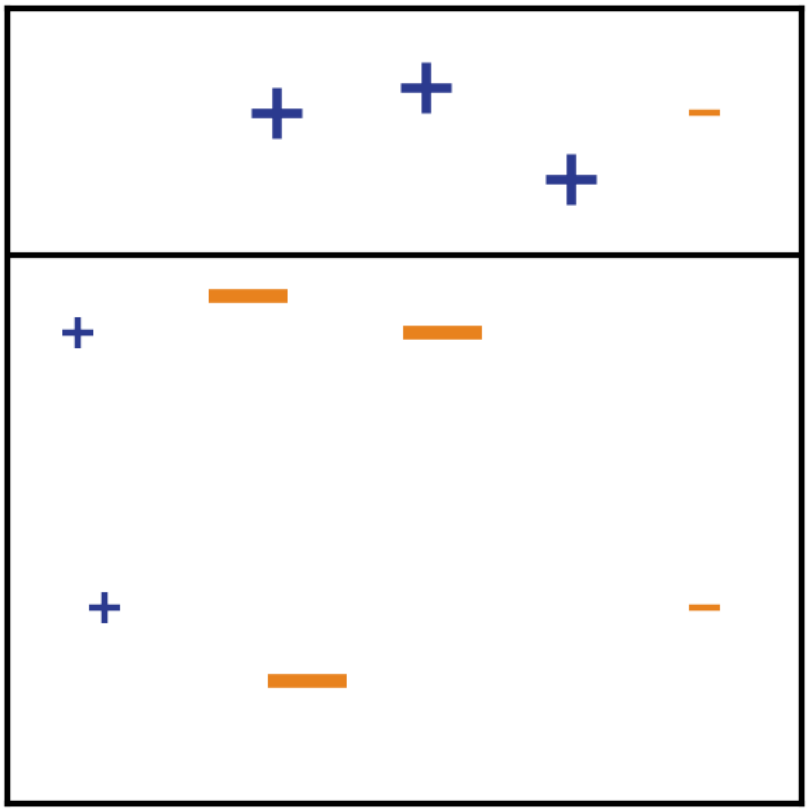
AdaBoost: Image 5



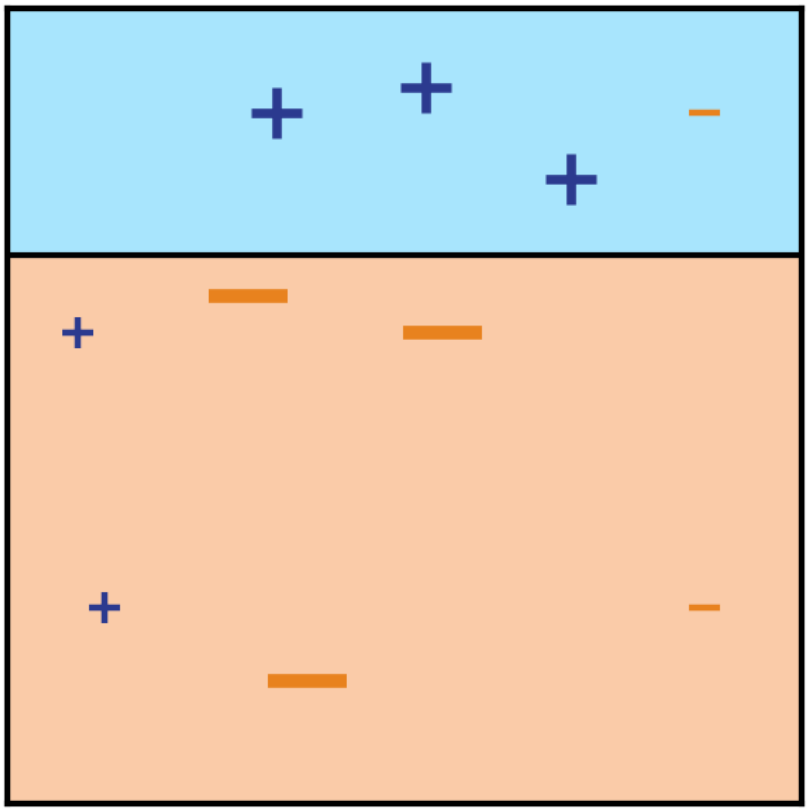
AdaBoost: Image 6



AdaBoost: Image 7

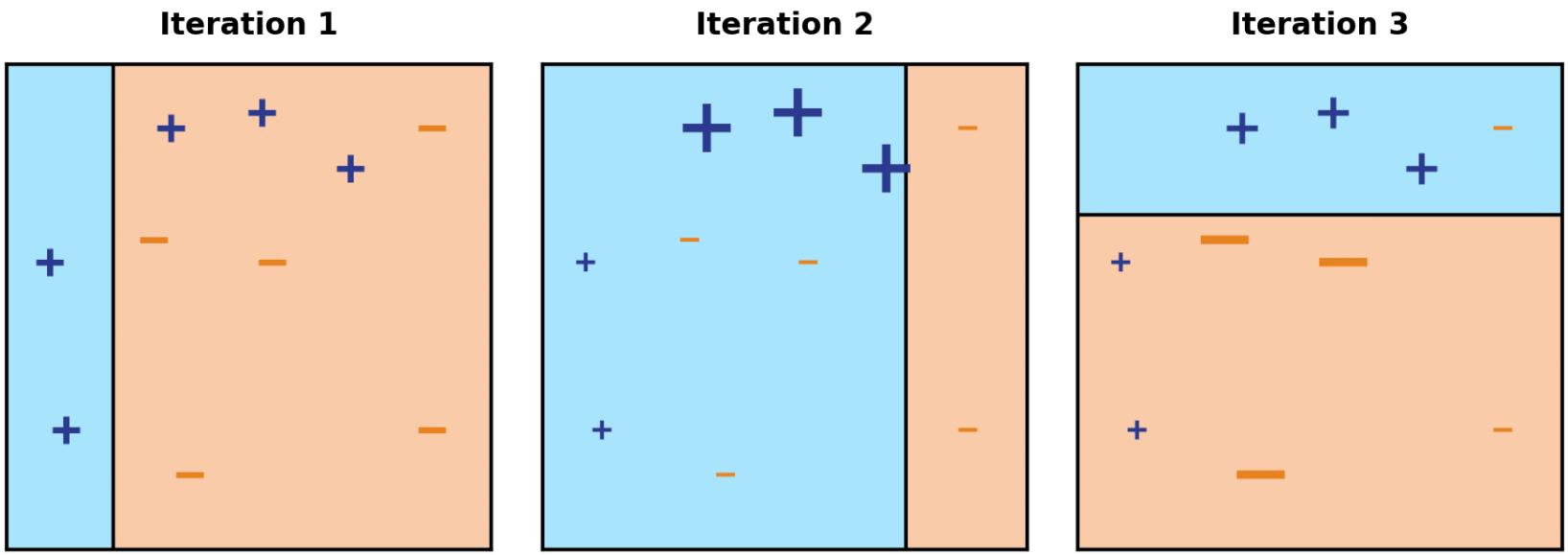


AdaBoost: Image 8



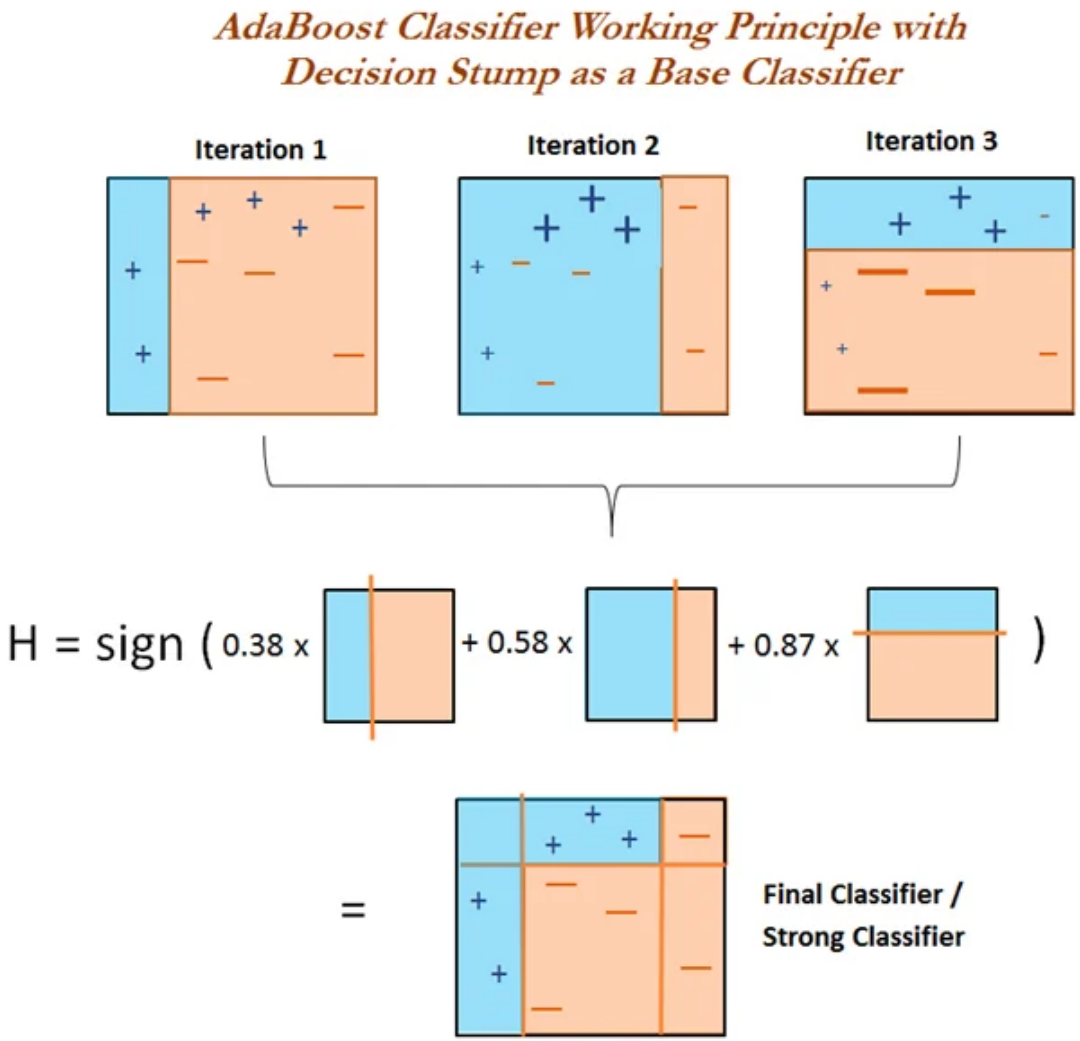
AdaBoost: Image 9

Recap of 3 iterations of sequence:



1. 3 + are missclassified, thus get more weight next
2. 3 - are missclassified, thus get more weight next
3. 2 + and 1 - are missclassified, thus get more weight next

AdaBoost: Overall



AdaBoost: Model

In AdaBoost, $G()$ is a sum of M sequentially built base models trained on differently weighted versions of the data using the Sign function:

$$G(x) = \text{Sign}\left(\sum_{m=1}^{M} \alpha_m G_m(x) \right)$$

$\alpha_1, \ldots, \alpha_M$ are weights ≥ 0 that allow each base model to contribute differently to the final prediction, thereby allowing some models to contribute more.

AdaBoost: Model

Each training data-point, $(\mathbf{x}_i, y_i)$ is also given a different weight, w_{im}, at each iteration

- At the first step of the algorithm, these weights are set to $\frac{1}{n}$ i.e. equal
- At iteration m the weights of observations that are:
  - misclassified by $G_m(x)$ are increased by a factor of $\exp(\alpha_m)$
  - classified correctly are left unchanged, thus weights *decrease relative* to the others

In today's lab

Three tuning parameters control AdaBoost, and the lab explores all of them:

- **Learning rate** (`learning_rate`): shrinks how much each tree contributes. Smaller values learn more slowly, so they need more trees
- **Number of boosting iterations** (`n_estimators`, i.e. the number of trees): too few underfit, too many can overfit. You'll tune this together with the learning rate using `GridSearchCV()`, then plot accuracy against the number of iterations
- **Complexity of the base trees** (`max_depth`): by default AdaBoost uses stumps (depth 1). You'll compare depth 1 against depth

AdaBoost: Algorithm Pseudocode

```
for i in 1:n
  # initialize equal weight for each point
  w_i = 1/n

for m in 1:M
  # Fit base model using weights
  G_m = model.fit(X, y, w)
  # Calculate weighted error of base model
  err_m = sum(w_i * I(y_i != G_m(x_i))) / sum(w_i)
  # Calculate overall base model weight
  alpha_m = learning_rate * log((1-err_m)/err_m)

  # Reweight each point: misclassified points get heavier
  for i in 1:n
    w_i = w_i * exp(alpha_m * I(y_i != G_m(x_i)))

# Final prediction: weighted vote of all M base models
G(x) = sign(sum(alpha_m * G_m(x)))
```

AdaBoost (algorithm)

- Each model's contribution $\alpha_m = \log\left((1-\text{err}_m)/\text{err}_m\right)$ in the ensemble is based upon its accuracy. Examples:
  - $\text{err}_m = 0.1$: $\alpha_m \approx 2.20$. Very accurate, so a big say in the final vote. Misclassified points get 9 times the weight
  - $\text{err}_m = 0.3$: $\alpha_m \approx 0.85$. Moderately accurate, so a moderate say. Misclassified points get about 2.3 times the weight
  - $\text{err}_m = 0.5$: $\alpha_m = 0$. No better than a coin flip, so no say in the final vote. Weights don't change

Gradient Boosting

- AdaBoost re-weights observations that are misclassified before training the next model
- Gradient boosting takes a more general approach: train subsequent models to the *residuals* (or loss contributions for cost functions other than squared error)

Gradient Boosting vs AdaBoost (diagram)

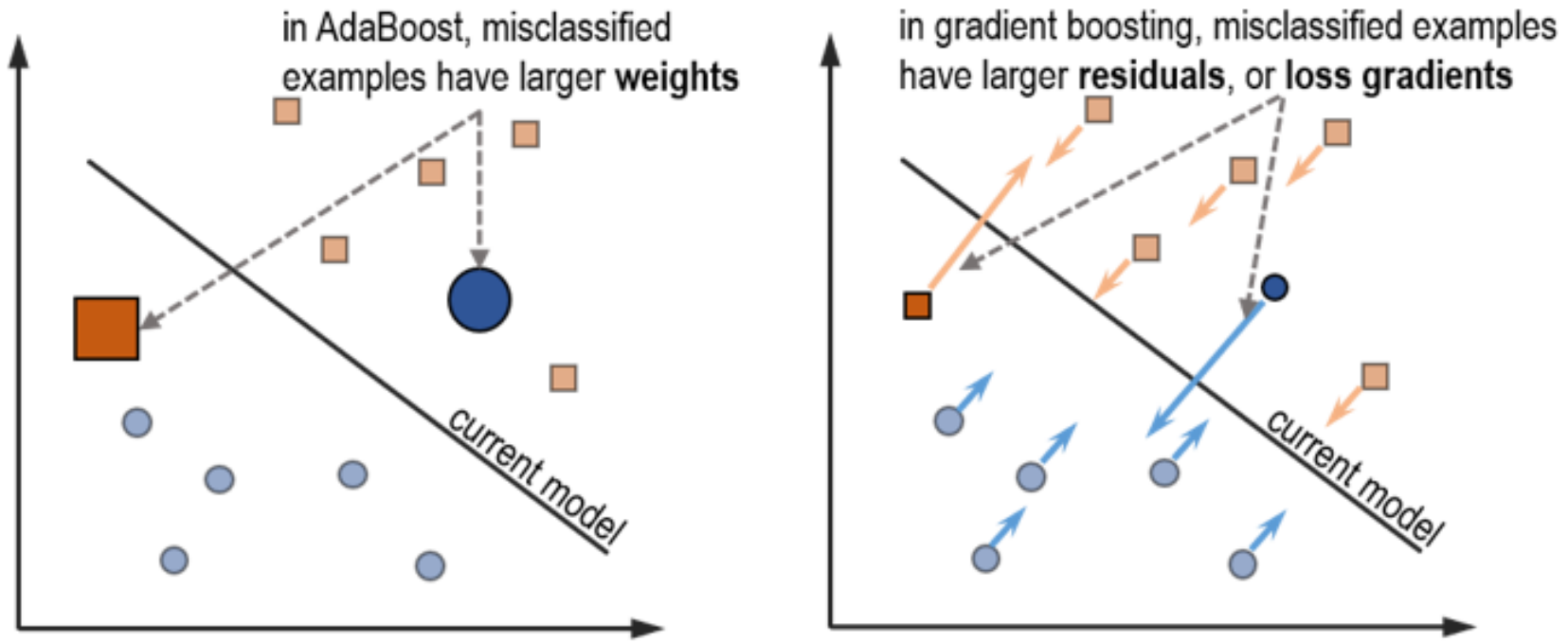


Image credit: Ensemble Methods for Machine Learning (textbook)

Gradient Boosting (pseudocode)

```
# Initialize residuals
r = y
# Initialize a blank model
f = 0

for m in 1:M:
    # Fit model to residuals
    f_m = model.fit(X, r)
    # Add new model to ensemble
    f = f + alpha * f_m
    # Update residuals
    r = r - alpha * f_m.predict(X)
```

- The output is f an ensemble model consisting of M base models
- The learning rate α is a small positive number that controls how quickly boosting learns (by limiting how much a model can contribute to the ensemble)

Gradient Boosting (comments)

- Unlike bagging (random forests), boosting can overfit if too many base models are used, so the number of boosting iterations should be careful chosen.
- The learning rates (value of α) and the number of boosting iterations work jointly to dictate whether a model ends up being overfit or underfit
- Like random forests, boosting tends to work best using shallow simple decision trees (max depths of either 1 or 2)

xgboost

- The `xgboost` library is a popular implementation of gradient boosting
  - It was developed by Tianqi Chen and Carlos Guestrin in 2016
  - The name stands for "Extreme Gradient Boosting"
- `xgboost` was designed to be able to run a GPU using parallelization and accommodate sparse data matrices (a data structure we will not discuss)
- In today's lab: you'll play with `xgboost`, tuning the 1) depth of its trees, 2) learning rate and 3) number of trees, and finally checking the best model's accuracy and F1 score on the test set